PYTHON TUTORIAL

Merging Text Files

A guide to combining data sources, appending content, and managing file formats.

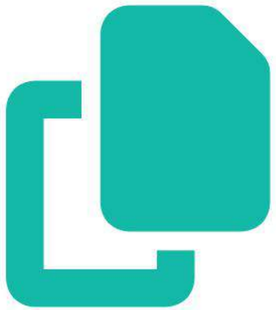
Prepared for Python Learners

The Challenge

Goal

We have two text files: `first.txt` and `second.txt`. We need to combine their contents into a new file `merged.txt`.

- **Order Matters:** File 1 contents first, then File 2.
- **Formatting:** Maintain the list structure.
- **Cleanliness:** Avoid "unwanted blank lines" between the datasets.



Input vs. Output

first.txt

```
one
two
three
```

second.txt

```
four
five
```

merged.txt

```
one
two
three
four
five
```

→ Combined Sequence

The "Newline" Trap

The Missing Link

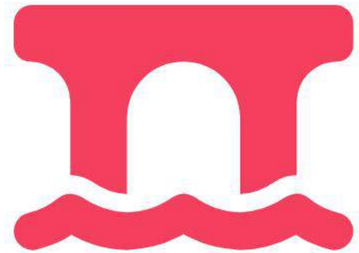
When files are merged, the transition point is critical.

Does `first.txt` end with a newline?

If NOT, simple concatenation results in data collision.

`three` + `four` = **threefour**

`three\n` + `four` = **three**
four



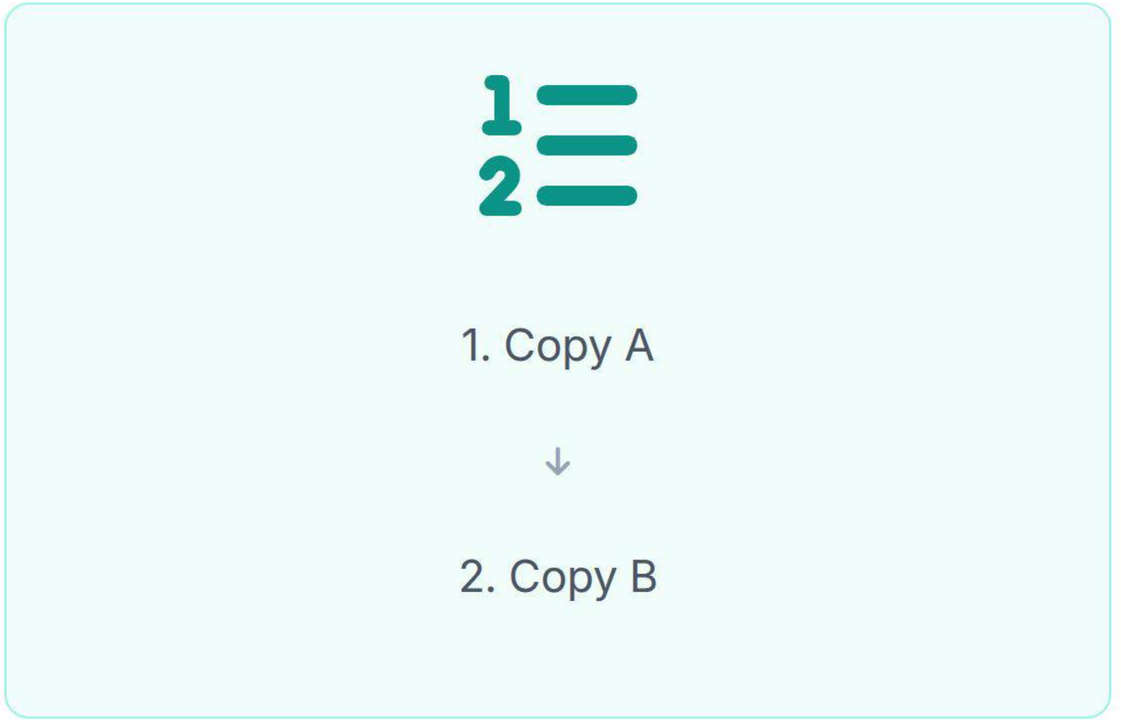
Solution 1: The Appending Loop

Sequential Processing

We perform the write operation in two distinct phases.

- **Step 1:** Read File A line-by-line and write to Output.
- **Step 2:** Manually ensure a newline exists.
- **Step 3:** Read File B line-by-line and write to Output.

This is memory efficient for large files.



Flowchart: Sequential Copy

Open

Create merged.txt

Copy A

Loop & Write lines

Check \n

Ensure line break

Copy B

Loop & Write lines

Close

Save file

Code: Sequential Approach

```python
def merge_files_sequential():
    filenames = ['first.txt', 'second.txt']

    with open('merged.txt', 'w') as outfile:
        for fname in filenames:
            with open(fname) as infile:
                # Read file content
                content = infile.read()

                # Write content
                outfile.write(content)

                # Ensure newline between files
                # (Prevent "threefour" collision)
                if not content.endswith('\n'):
                    outfile.write('\n')
```

Code Breakdown

- `outfile.write(content)`: Writes the entire block of text.
- `content.endswith('\n')`: The safety check. If the file didn't end with a newline, we add one manually so the next file starts on a fresh line.

Solution 2: Clean Concatenation

The "Strip & Join" Method

To strictly enforce "no unwanted blank lines," we can strip the padding from both files and insert exactly one newline between them.

- **Strip:** Remove leading/trailing whitespace from File A and File B.
- **Join:** Combine them with exactly one `\n`.
- **Result:** Perfect formatting guaranteed.

Flowchart: Clean Merge

1. Read All

Load both files into variables: `text1`, `text2`.

2. Sanitize

`text.strip()` removes extra newlines/spaces from ends.

3. Merge

`text1 + "\n" + text2` guarantees 1 divider.

Code: Clean Merge Solution

```
def merge_files_clean():
    try:
        # Read both files completely
        with open('first.txt') as f1, \
             open('second.txt') as f2:

            data1 = f1.read().strip()
            data2 = f2.read().strip()

        # Write with exact separator
        with open('merged.txt', 'w') as out:
            out.write(data1 + "\n" + data2)

    except FileNotFoundError:
        print("Files missing.")
```

Code Breakdown

- `read().strip()`: Removes any accidental blank lines at the start or end of the source files.
- `+ "\n" +`: This is the "glue." It ensures there is exactly one newline between the two datasets, no more, no less.

Handling Edge Cases

Scenario: Empty Files

What if `first.txt` is empty?

Result (Sol 2): You might get a leading newline (e.g., `"\n" + data2`). Ideally, we should check if data exists before writing.

Fix:
```
content = [data1, data2]
final = "\n".join([x for x in content if x])
```

Visualizing Blank Input

```
File 1: [Empty]
File 2: "Hello"
---
Naive Merge: "\nHello" (Blank top line)
Smart Merge: "Hello"
```

Comparing Approaches

Feature	Solution 1 (Append)	Solution 2 (Clean/Strip)
Formatting Control	Harder (Must check `endswith`)	**Perfect** (Exact control)
Memory Usage	**Low** (Can stream lines)	High (Loads whole files)
Code Simplicity	Verbose	**Concise**
Risk	"threefour" collision	Accidental blank lines (if empty)